DAY 04 | HANDS-ON ACTIVITIES

Architecture and MongoDB: Problem Statements

Seven problems. Colab users: use mongomock as in the notebook. VS Code users: use a local MongoDB or a free Atlas cluster.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Expense Chart

Level: Warm-up | **Concepts:** Matplotlib bar chart

Given categories ["Food", "Rent", "Travel", "Fun"] and amounts [6000, 15000, 2500, 3000], draw a bar chart with a title, axis labels and the value on top of each bar.

Sample run / expected result

```
A bar chart titled "Monthly expenses" with 4 labelled bars
```

Hints

- plt.bar returns bars; plt.text(x, y, value) places labels.

Problem 2: Trend Line

Level: Easy | **Concepts:** line chart, two series

Plot website visitors for 12 months for two years on the same chart with a legend. Save it as visitors.png.

Sample run / expected result

```
visitors.png with two coloured lines and a legend
```

Hints

- Call plt.plot twice with label=...
- plt.legend(); plt.savefig("visitors.png")

Problem 3: Architecture Sketch

Level: Easy | **Concepts:** client server, layers

For a food delivery app, write down the client(s), the API endpoints you would need (at least 5), the server layers, and the collections in the database. No code needed.

Sample run / expected result

```
Clients: customer app, restaurant app
Endpoints: GET /restaurants, POST /orders, ...
Collections: users, restaurants, orders
```

Hints

- Think about what each screen in the app needs.

Problem 4: Product Catalogue: Create and Read

Level: Medium | **Concepts:** insert_many, find, filters

Insert 8 products with name, category, price and stock. Print all products under 1000, all electronics, and the count of products with stock below 5.

Sample run / expected result

```
Under 1000: Pen, Notebook, Mug ...
Electronics: Mouse, Keyboard, Monitor
Low stock: 2
```

Hints

- {"price": {"$lt": 1000}}
- count_documents(filter)

Problem 5: Product Catalogue: Update and Delete

Level: Medium | **Concepts:** update_one, update_many, delete

Increase the price of all electronics by 10 percent, mark products with stock 0 as {"active": False}, then delete inactive products. Print the collection after each step.

Sample run / expected result

```
After update: Mouse 550.0 ...
Deleted: 1
```

Hints

- Real MongoDB supports {"$mul": {"price": 1.1}}. mongomock does not, so loop over find() and use $set.

Problem 6: Category Report

Level: Challenging | **Concepts:** MongoDB read + Matplotlib

Read all products, calculate total stock value (price x stock) per category in Python, and draw a pie chart of the share of value per category.

Sample run / expected result

```
Pie chart with Electronics 62%, Stationery 18%, Home 20%
```

Hints

- Accumulate totals in a dict, then plt.pie(values, labels=keys, autopct="%1.0f%%").

Problem 7: Repository Class

Level: Stretch | **Concepts:** OOP + MongoDB, layered design

Write a ProductRepository class with add, get_by_name, list_by_category, update_price and delete methods. The rest of your code should only call the repository, never the collection directly.

Sample run / expected result

```
repo = ProductRepository(db)
repo.add({...})
repo.list_by_category("Home")
```

Hints

- Pass the collection into __init__ so it can be swapped for a real database later.

Ganbatte (頑張って): do your best